

1. Introducción

Esta práctica se basa en el código que ya teníamos del proyecto anterior (el mundo con arena, los murciélagos y el gladiador). La idea era tomar esa misma estructura de clases y adaptarla para hacer un juego completamente diferente.

Lo que se propuso fue algo parecido a Animal Crossing, aunque no una copia del juego. Nada de deudas ni tiendas ni nada de eso. La idea es: un mundo tranquilo con hierba, árboles y un lago donde puedes caminar y encontrarte con aldeanos amigables que te saludan. También hay objetos regados por el mapa que puedes recoger.

No se cambiaron las clases base (Sprite, TileSet, Tile) porque funcionan igual. Lo que si se modificó fueron las clases que dependen de ellas y se agregaron dos completamente nuevas.

2. Descripción del videojuego

El juego se llama *Mi Aldea*. Es un mundo de 5000 x 5000 pixeles con cámara que sigue al jugador, igual que en la práctica anterior. La diferencia principal es el ambiente: en vez de una arena con columnas y murallas hay un pueblo con caminos de tierra, bosques de árboles, un lago y parches de flores.

El jugador puede moverse con las flechas del teclado, correr con Shift y recoger objetos del suelo presionando la barra espaciadora. Los objetos que se pueden recoger son manzanas, naranjas, flores, conchas y estrellas de mar. Cuando se recoge algo aparece una notificación con el nombre del objeto y se guarda en el inventario que se ve en la parte de abajo de la pantalla.

Los NPCs son los aldeanos: cinco simples que caminan por zonas del mapa y se acercan a saludar cuando el jugador pasa cerca, y un habitante especial llamado Toni-moqui que usa el algoritmo A* para llegar hasta el jugador esquivando obstáculos.

También hay un ciclo de día y noche. El color del cielo cambia lentamente entre noche, amanecer, día, atardecer y crepúsculo. Es un ciclo visual nada más, no afecta la jugabilidad.

2.1 El mapa

El mapa tiene cinco tipos de terreno. La hierba verde cubre la mayor parte del mundo y es por donde el jugador puede moverse libremente. Los caminos de tierra forman una cruz que pasa por el centro del mapa; son visualmente mas oscuros que la hierba y el algoritmo A* los trata como terreno lento (costo doble), lo que hace que los NPCs que usan A* prefieran rodearlos si pueden.

Los arboles estan agrupados en bosques en las esquinas y los bordes del mapa; son obstaculos solidos, ni el jugador ni los aldeanos pueden atravesarlos. El lago esta ubicado en el cuadrante noreste y tambien es un obstaculo solido. Por ultimo hay manchas de flores dispersas que no bloquean el paso pero sirven de decoracion y a veces tienen items cerca.

Todo el mapa se dibuja una sola vez al inicio del programa y se guarda en una sola superficie grande. Eso hace que el juego cargue rapido aunque el mapa sea enorme, porque durante el juego solo se hace un blit de esa superficie ya preparada.

2.2 El personaje (jugador)

El personaje es el mismo sprite (monitoB.png), un monito animado que camina en cuatro direcciones. Se controla con las flechas del teclado. Manteniendo Shift presionado el personaje corre, lo que duplica su velocidad. La cámara siempre centra al personaje en la pantalla, igual que en la práctica anterior.

Lo nuevo en esta práctica es el inventario. El personaje puede cargar hasta 10 objetos. Para recoger algo hay que acercarse a un ítem y presionar la barra espaciadora; el ítem desaparece del mapa y aparece en los slots de la barra inferior de la pantalla. Cuando se recoge algo también aparece una notificación breve con el nombre del objeto en el centro de la pantalla.

El personaje tiene tres estados internos que determinan qué animación se reproduce: quieto cuando la velocidad es cero, caminando cuando se mueve sin Shift, y corriendo cuando Shift está activo.



2.3 Los aldeanos

Hay cinco aldeanos distribuidos en distintas zonas del mapa. Sus nombres son Pico, Luna, Mochi, Dali y Benji. Cada uno tiene un color distinto para poder distinguirlos fácilmente. Se dibujan con código usando formas geométricas: cabeza redonda, orejas triangulares de animal, cuerpo ovalado y piernas que se mueven cuando caminan.

Su comportamiento funciona con una máquina de tres estados. En el estado DEAMBULAR, que es su estado normal, el aldeano elige un punto aleatorio dentro de su zona asignada, camina hacia ese punto, espera unos segundos ahí, y vuelve a elegir otro destino. Esto hace que parezca que tienen vida propia aunque en realidad solo estén siguiendo lógica simple.

Cuando el jugador se acerca a menos de 220 píxeles de distancia, el aldeano entra al estado SALUDAR. Se mueve lentamente hacia el jugador y muestra una burbuja de diálogo con una frase aleatoria como "Hola viajero!" o "El lago se ve precioso." Después de un momento (o si el jugador se aleja), el aldeano cambia al estado REGRESAR y vuelve caminando a su posición de origen, tras lo cual retoma el estado DEAMBULAR.

Los aldeanos simples no pueden rodear obstáculos. Si hay un árbol en su camino, simplemente paran y esperan hasta que su temporizador genera un nuevo destino. Eso se nota especialmente cerca de los bosques, donde a veces quedan "pegados" contra un árbol por un momento antes de reencaminarse.



2.4 Habitante especial: Toni - mon

Es el sexto NPC del juego y visualmente se ve igual que los otros aldeanos (mismo sistema de dibujo procedural), pero su comportamiento es diferente. Tiene su zona asignada en la parte centro-este del mapa y cuando detecta al jugador a menos de 260 píxeles calcula una ruta usando el algoritmo A*, igual que el Gladiador del proyecto anterior.

La diferencia practica se nota en el movimiento: mientras que los aldeanos normales van en linea recta y se traban contra los obstaculos, Toni siempre encuentra un camino aunque tenga que rodear un bosque entero o bordear el lago. Recalcula la ruta cada 20 frames por si el jugador se mueve.

Tambien usa A* para regresar a su base cuando el jugador se aleja. En el HUD de depuracion que aparece en la pantalla se puede ver en que estado esta Toni en cada momento: DEAMBULANDO, SALUDANDO o REGRESANDO.

2.5 Objetos coleccionables

Al inicio del juego se colocan 40 objetos dispersos por el mapa en posiciones aleatorias. Solo se ubican en terreno libre (hierba, caminos o flores), nunca dentro de un obstaculo ni en el lago. Los tipos de objeto que existen son:

- Manzana: un circulo rojo con un tallo verde. Es el objeto mas comun.
- Naranja: un circulo de color naranja con brillo, igual que la manzana pero de diferente color.
- Flor: una figura de cinco petalos rosas con un centro amarillo claro.
- Concha: una elipse de color crema con lineas de arco para simular la textura de una concha de mar.
- Estrella de mar: un poligono de diez puntos en color amarillo.

Todos los items tienen una animacion de brillo: un halo blanco semitransparente que aparece y desaparece lentamente para que sean faciles de ver en el mapa. Una vez que el jugador recoge un item, desaparece para siempre hasta que se reinicia el juego. No hay reaparicion de items en esta version.

3. Imágenes y recursos

3.1 Imágenes reutilizadas del proyecto anterior

Se reutilizaron dos archivos de imagen que ya teníamos:

- monitoB.png: el Sprite del personaje principal. No se cambió nada, funciona exactamente igual que antes.
- Arena_Tileset.png: se usa como base para dibujar el suelo del pueblo. Las filas 8 y 9 del tileset son los tiles de pasto/arena que se usan como hierba, y las filas 13 a 16 se usan para los caminos de tierra.

3.2 Gráficos generados por código

Los aldeanos, los árboles, el lago, las flores y los objetos recogibles se dibujan directamente con funciones de pygame (pygame.draw.circle, polygon, rect, ellipse). No necesitan archivos de imagen. Esto para no depender de recursos externos y que el juego funcione en cualquier sin buscar sprites adicionales.

3.3 Recursos opcionales para mejorar el juego

Si en el futuro se quisieran agregar sprites reales en vez de los generados por codigo, las mejores opciones gratuitas son kenney.nl, opengameart.org buscando "LPC tileset", e itch.io buscando "free pixel art village".

4. Estructura del código

El proyecto se divide en dos archivos igual que antes: Sprites_pueblo.py con todas las clases y mundo_pueblo.py con el bucle principal del juego.

4.1 Clases sin cambios

Estas tres clases se copiaron directamente del proyecto anterior porque no había razón para modificarlas:

- Sprite: la clase base que carga la imagen y define el rectángulo de colisión. Solo se le agrego un if para que no falle si el archivo de imagen no existe.
- TileSet: hereda de Sprite, carga el tileset y extrae los frames individuales.
- Tile: representa una celda del mapa con su posición, tipo e imagen. Igual que antes.

También se copió la función `_astar()` sin ningún cambio. El algoritmo es el mismo, con soporte para tiles de costo variable (tipo 1 = camino lento, costo x2).

4.2 TileSetPueblo

Esta clase hereda de TileSet igual que TileSetArena, pero genera un mapa de aldea en vez de una arena. La lógica interna es similar: se construye una cuadrícula, se asigna un tipo a cada celda y se dibujan los tiles encima.

Los tipos de tile son:

- 0 = hierba (libre),
- 1 = camino de tierra (costo x2 en A*),
- 2 = árbol (obstáculo),
- 3 = agua (obstáculo) y
- 4 = flores (decorativo, pisable).

El mapa se construye en cinco pasos: primero se llena todo de hierba, luego se traza una cruz de caminos, después se colocan grupos de árboles en distintas zonas, se genera un lago elíptico en el cuadrante noreste y por último se dispersan parches de flores aleatorios.

4.3 Aldeano

El Aldeano es el equivalente al Murcielago del proyecto anterior pero amigable. Tiene la misma maquina de estados de tres estados: DEAMBULAR, SALUDAR y REGRESAR. No hereda de TileSet porque se dibuja con código y no necesita cargar una imagen.

4.4 Habitante

El Habitante hereda de Aldeano igual que el Gladiador heredaba del Murcielago. La diferencia es que los estados SALUDAR y REGRESAR usan A* para moverse, entonces puede rodear obstaculos correctamente. Recibe el mapa_celdas como parametro para que A* conozca los costos de cada tipo de tile.

4.5 Item (clase nueva)

Esta clase es completamente nueva. Un Item es un objeto recogible (manzana, naranja, flor, concha o estrella de mar) con animacion de brillo. El jugador lo recoge presionando ESPACIO cuando esta cerca. Cada tipo se dibuja con una forma diferente usando pygame.draw.

4.6 Personaje

El Personaje es basicamente el mismo que antes. Se le agregaron: una lista inventario (maximo 10 objetos), la variable recoger_activo que se activa cuando se presiona ESPACIO, y el metodo recoger_item() que revisa si hay algun item cercano y lo mete al inventario.

5. Diagramas

En esta seccion se incluyen los diagramas que describen la estructura y el comportamiento del programa. El diagrama de clases muestra como se relacionan las clases, las maquinas de estado describen el comportamiento de cada clase con estados, y el diagrama de flujo muestra como corre el juego.

5.1.1 Diagrama general

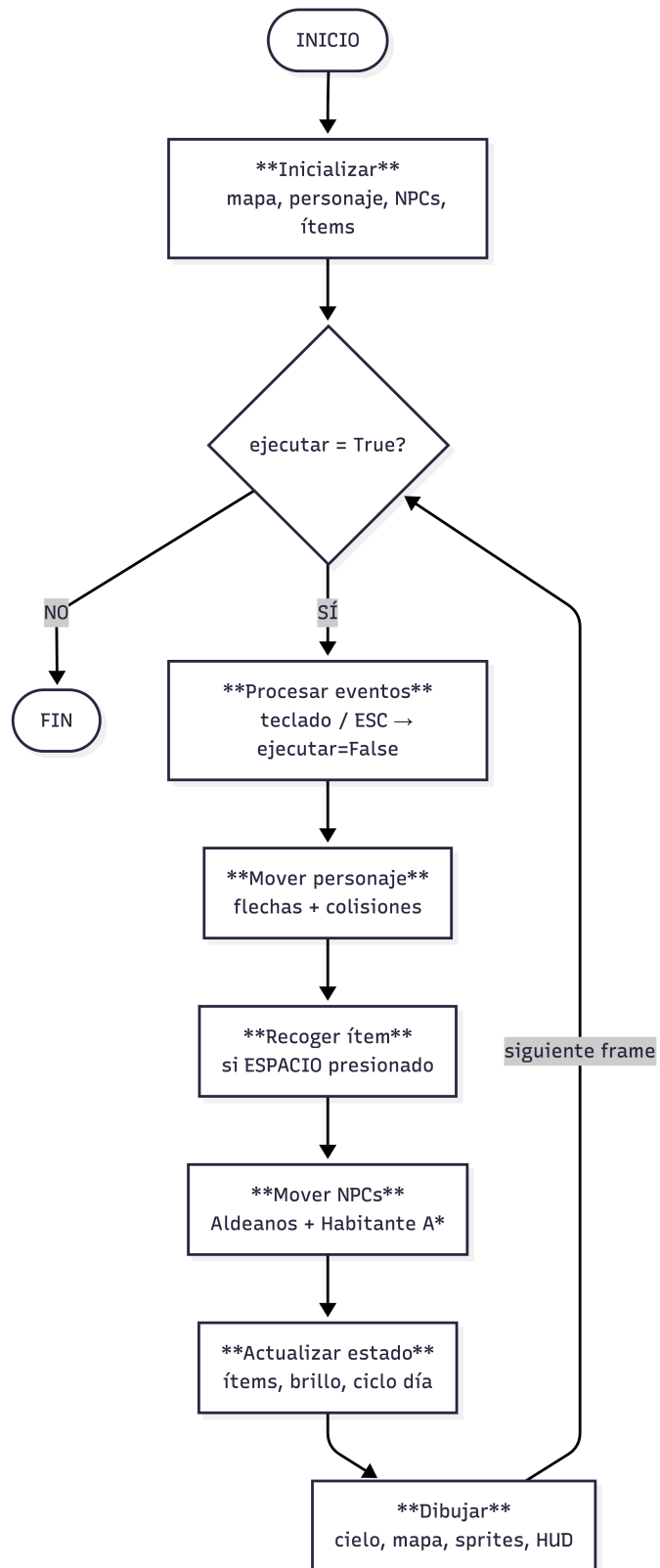


Figura 1. Diagrama de flujo general del bucle principal del juego

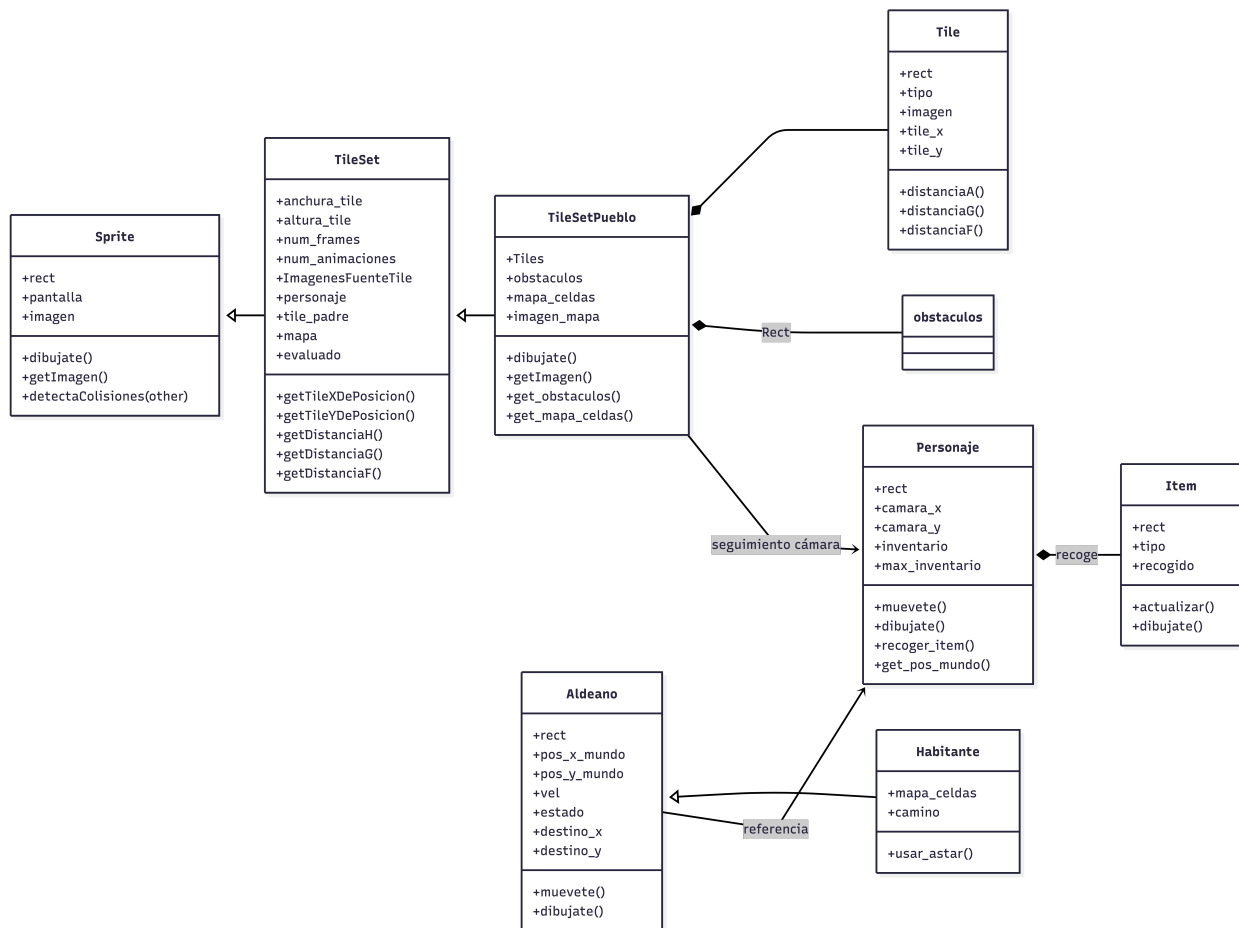


Figura 1.1 Diagrama de clases

5.1 Maquinas de estado

Las siguientes figuras muestran las maquinas de estado para las clases que tienen comportamiento por estados. Aldeano y Habitante comparten la misma estructura de tres estados, aunque Habitante usa A* internamente.

5.2.1 Aldeano

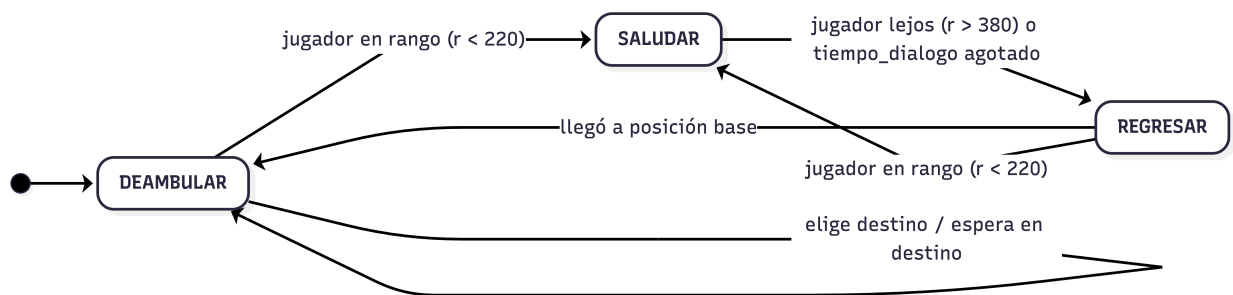


Figura 2.1. Maquina de estados — clase Aldeano

5.2.2 Habitante

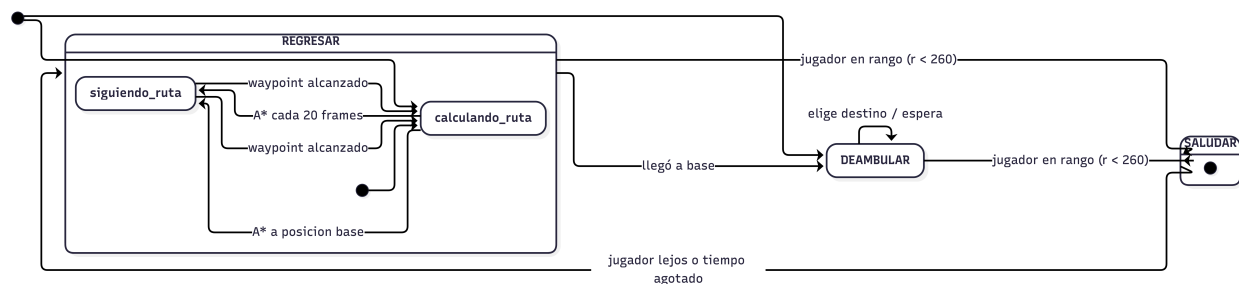


Figura 2.2 Máquina de estados — clase Habitante (con A* en SALUDAR y REGRESAR)

5.2.3 Personaje

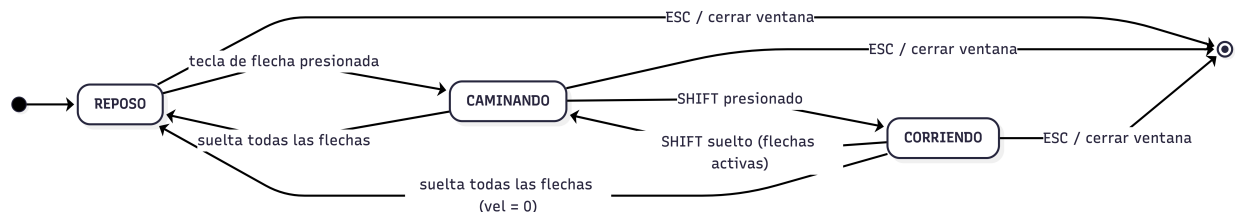


Figura 2.3. Máquina de estados — clase Personaje

5.2.4 Ítem

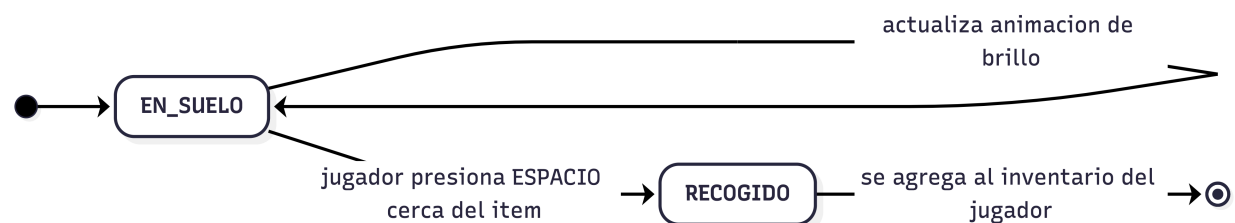


Figura 2.4. Máquina de estados — clase Ítem

6. Dificultades encontradas

La primera fue que los Aldeanos heredaran de SpriteAnimado para mantener la misma cadena de herencia del proyecto anterior. El problema es que SpriteAnimado hereda de TileSet y TileSet necesita cargar un archivo de imagen. Como los aldeanos se dibujan con código no hay imagen que cargar, y cada que intentaba crear un aldeano tronaba con un error de que no encontraba el archivo.

En Python no es obligatorio seguir exactamente la misma cadena de herencia. Si la clase nueva no necesita lo que da la clase padre, es mejor no heredar de ella y solo duplicar lo que si se necesita. Por eso Aldeano no hereda de nada, tiene sus propios atributos de posición y velocidad directamente.

La segunda fue el tiempo de carga del mapa. Con el mapa de 5000x5000 y todos los tiles dibujados individualmente, al principio tardaba como 20 segundos en cargar. El problema era que estaba llamando `getTileHierba()` dentro del ciclo de construcción y esa función usa `randint` cada vez, lo cual no es el problema real. El verdadero cuello de botella era hacer `Surface.blit()` miles de veces en el ciclo principal en vez de una sola vez al inicio. Cuando se cambió para pregenerar el mapa completo en un solo `Surface` al inicio, el tiempo de carga bajo a menos de 3 segundos. Eso es lo que hace `imagen_mapa`.

Otro detalle menor fue que los diálogos de los aldeanos tienen caracteres especiales como acentos. En Windows a veces esto causa problemas dependiendo del encoding del archivo. La solución fue simplemente quitarles los acentos a las frases del dialogo para evitar errores.

7. Conclusión

Esta práctica me ayudo a entender para que sirve la herencia en programación orientada a objetos. Es poder reutilizar comportamientos que ya funcionaban. El ciclo de día, los aldeanos y el sistema de inventario son cosas que no existían antes, pero el movimiento del personaje, el A* y la cámara funcionan igual que en el proyecto anterior.

La diferencia entre Aldeano y Habitante: los dos hacen lo mismo visualmente, pero uno llega a su destino en línea recta y el otro rodea los obstáculos.

Se puede agregar música de fondo y que los aldeanos digan cosas diferentes según la hora del día. También se podría actualizar a los ítems para reaparecer con el tiempo y que el mapa no se quede vacío.